

TEU



FCFS

P.E.N.P

ROUND-ROBIN

“AVALIAÇÃO COMPARATIVA DE POLÍTICAS DE ESCALONAMENTO” E PROPOSIÇÃO DO ALGORITMO TEJ T.E.J?

Algoritmo Triagem com Espera Justa

Equipe: André Wesley, Leôncio Ferreira, Paulo Gabriel, Salomão Rodrigues

Professor: Weskley Vinicius Fernandes Mauricio

Disciplina: Sistemas Operacionais

MODELO DE SIMULAÇÃO

- **Tempo Discreto (Ticks):**

O simulador não utiliza o relógio real do hardware, mas avança em unidades discretas indivisíveis chamadas ticks. Isso garante 100% de reprodutibilidade experimental, garantindo que a mesma seed gere exatamente os mesmos eventos em qualquer computador.

- **Rajadas de CPU (Exclusividade):**

Apenas um processo pode ocupar o estado Running por vez. Ele consome sua "rajada" ininterruptamente (no caso dos não-preemptivos) ou até o fim do seu quantum (no caso do Round-Robin).

- **Operações de E/S (Paralelismo no estado Blocked):**

A Entrada/Saída foi modelada como dispositivos paralelos. Isso significa que múltiplos processos podem permanecer no estado Blocked simultaneamente (ex: um no disco, outro na rede) sem sofrer contenção ou fila. Eles apenas aguardam a própria rajada de E/S acabar para voltar ao estado Ready.

- **Custo de Troca de Contexto (Overhead):**

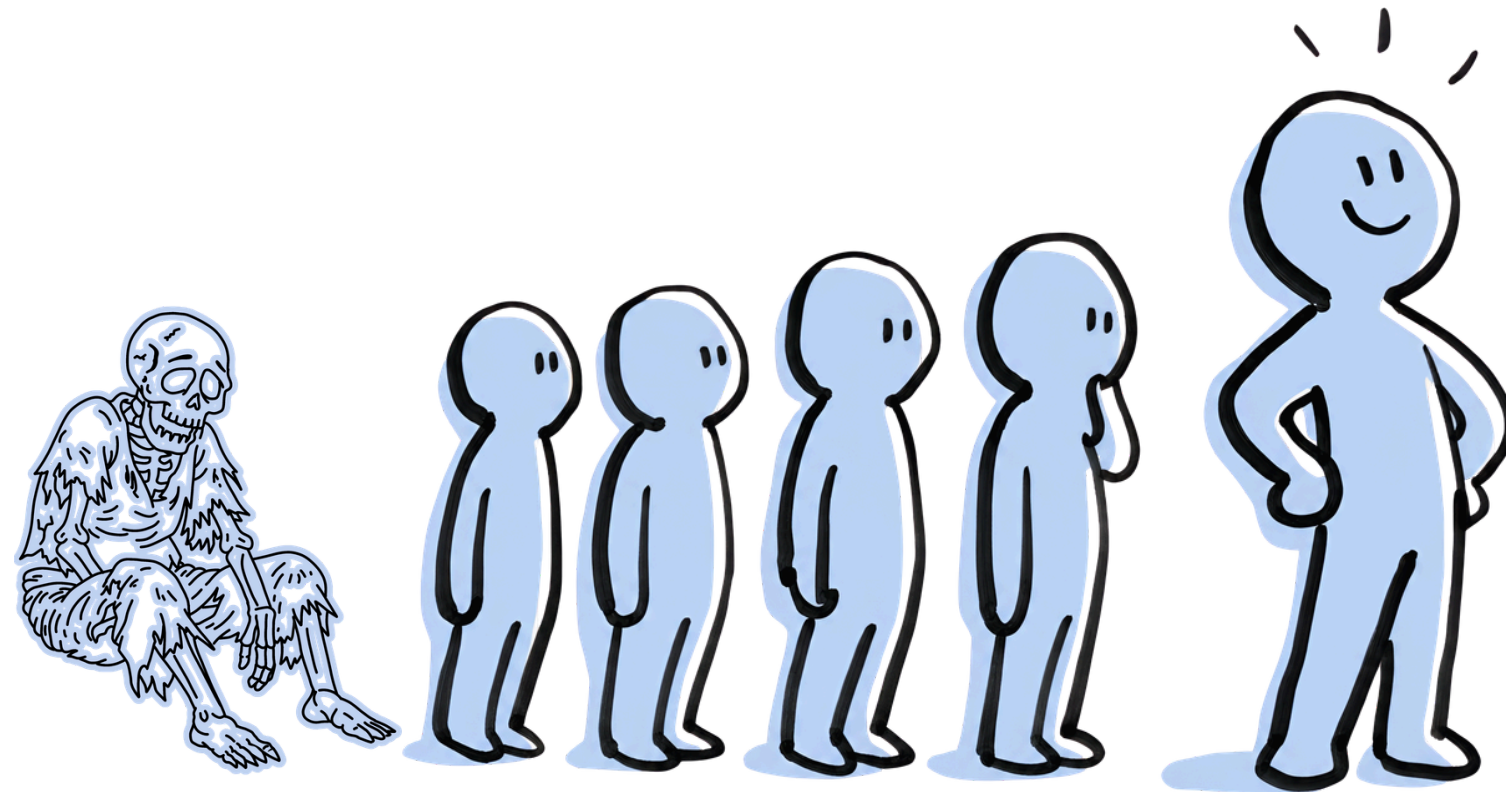
Sempre que um processo é despachado ou retirado da CPU, o sistema cobra um pedágio fixo de 1 tick ($\delta = 1$). Durante esse tempo exato, a CPU fica em estado de switching (totalmente ociosa/indisponível), simulando o tempo gasto pelo SO para salvar e restaurar o estado dos registradores na memória.

- **Ciclo de Vida de um Processo**



STARVATION (INANIÇÃO).

- "Algoritmos focados em **Prioridade Pura** rodam as coisas mais importantes primeiro."
- "Sob alta carga do sistema, tarefas com prioridade baixa nunca ganham a CPU."
- Isso gera a inanição (**starvation**). A tarefa fica esquecida na fila.

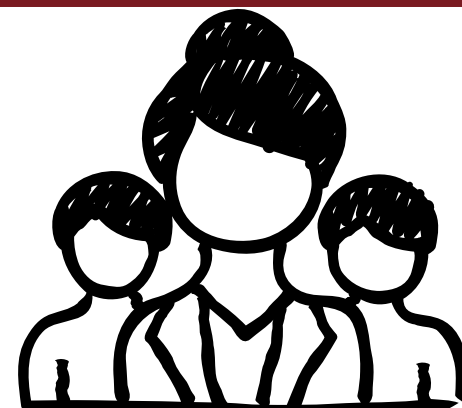
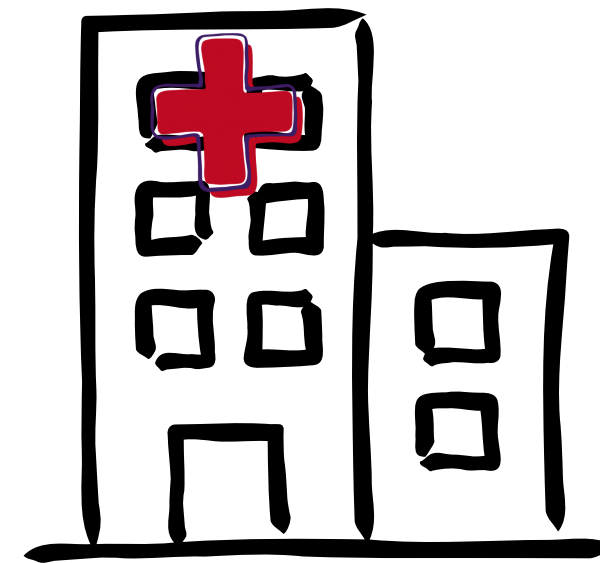
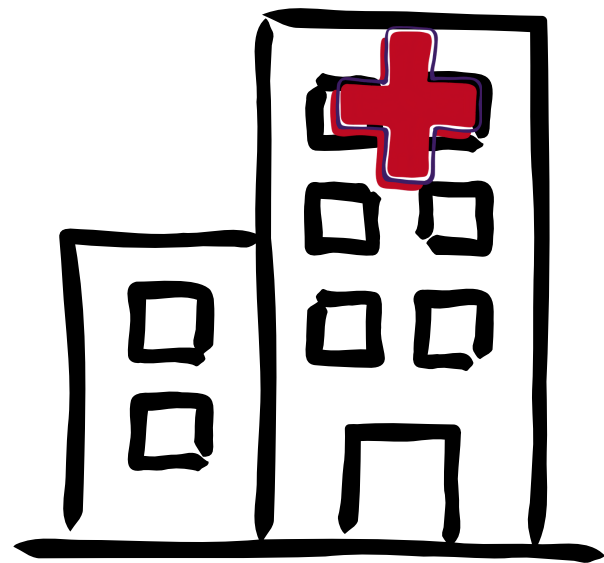


A NOSSA SOLUÇÃO - CONHEÇA O TEJ

Como forma de explicação vamos utilizar a analogia do **Protocolo de Manchester**. Onde, em um hospital, pacientes recebem pulseiras indicando a urgência de seus atendimento. **Verde → Pouco urgente** | **Vermelho → Emergência [entre outras cores]**

Protocolo de Manchester

- Pacientes com casos graves, recebem pulseira vermelha e pulam direto para o médico
- Pacientes com casos leves, recebem pulseira verde e esperam
- Se paciente com caso leve **esperar tempo demais** na recepção, o **sistema atinge o prazo máximo**. Ele é '**resgatado**' e vira **prioridade**.



COMO O TEJ FUNCIONA POR DEBAIXO DOS PANOS?

- Nos inspiramos no **Aging (Envelhecimento)** — que é a técnica clássica de aumentar a prioridade de quem espera muito tempo. Mas o nosso usa um limite matemático rígido (determinístico)."
 - Quando um processo entra na **Fila de Prontos (READY)**, calculamos um **Prazo de Resgate (rescue_deadline)** estático e inalterável.
-
- O algoritmo tem **duas etapas** para escolher a próxima tarefa:

Precedência Absoluta

Aguém estourou o prazo máximo de espera? Se sim, é "resgatado" e fura a fila.

Prioridade Normal

Ninguém estourou o prazo? Segue normalmente executando quem tem prioridade maior.

COMO O TEJ FUNCIONA POR DEBAIXO DOS PANOS?

- DETALHANDO MAIS

CrITÉRIOS de **Desempate Estritos**:

- 1. Fase do Algoritmo (Resgatados vencem normais)
 - 2. Tempo de chegada na Fila de Prontos (Ready)
 - 3. Menor ID do Processo
-
- Como o nosso escalonador **decide** exatamente **quem vai para a CPU**?
R: A menor prioridade numérica ($Pr = 0$) é **a mais importante**. Se houver **processos normais**, o **nível zero** entra primeiro.
 - Mas se **houver empate**?

R: Nós usamos a **causalidade estrita**: ganha **quem chegou primeiro na fila** de Prontos. Se por milagre eles entraram no mesmo exato milissegundo, **desempata pelo ID** de criação do processo.
 - Se um **processo** for '**resgatado**' na nossa triagem, ele **ignora** todas as **prioridades** e passa na frente de todo mundo.

COMO O TEJ FUNCIONA POR DEBAIXO DOS PANOS?

- DETALHANDO MAIS

Como saber se um processo é “velho” ?

- Fórmula do Limite de Espera (ΔW_i):
Limite = Intervalo de Resgate $\times$ (Prioridade do Processo + 1)
- A Condição de Resgate: Tick Atual // (Entrada na Fila + Limite)

- "Mas como o sistema sabe que a tarefa ficou 'velha' na fila?

Nós temos uma **fórmula matemática** justa baseada no **nível de prioridade**. Definimos o nosso **Intervalo de Resgate** Base como **10 ticks**. Se um processo tem Prioridade 0 (VIP), o limite dele é 10 ticks.

Se **ele tem Prioridade 10** (A mais baixa de todas), o **limite dele é 110 ticks** (10 vezes 11). A cada tick do relógio, o **algoritmo checa**: O momento **atual ultrapassou o tick** de **entrada** mais o **limite de espera**? Se a resposta for **SIM**, o processo **envelheceu demais**, o prazo estourou e ele **ganha precedência absoluta** para não morrer de inanição."

ARENA DE TESTES (COM QUEM COMPARAMOS?)

Slide: 9

- Fizemos **1.600 simulações** pesadas (100 sementes/seeds por cenário **garantindo o Intervalo de Confiança de 95%**).

Em **4 Cenários Diferentes**, comparando com **3 algoritmos diferentes**

Equilibrado

Uma mistura justa de processos curtos, longos, uso de CPU e de disco.

CPU-Bound

O cenário de estresse de processador. Tarefas matemáticas gigantescas que sequestram a CPU e quase nunca fazem pausas.

I/O-Bound

O cenário com muita Entrada/Saída. Processos rápidos que pausam toda hora para ler disco ou internet.

Prioridades Desbalanceadas

Um cenário caótico onde 85% das tarefas chegam como Alta prioridade. É aqui que os algoritmos entram em colapso e a inanição acontece de verdade.

FCFS (First-Come, First-Served)

Quem chega primeiro, usa a CPU até acabar sua tarefa (ou ir fazer Entrada/Saída)

Prioridade Estática Não Preemptiva

O escalonador olha quem tem a maior prioridade e roda até o fim da rajada.

Round-Robin (RR)

Ele dá uma fatia de tempo exata (um quantum de 10 ticks) para cada processo. Se não terminar nesse tempo, o processo é "arrancado" da CPU à força (preempção) e vai para o final da fila.

TURNAROUND

- **O que é:** É o tempo total de vida do processo no sistema. O cronômetro liga no tick em que o processo chega/nasce, e só desliga no tick em que ele conclui 100% da sua tarefa.
- **Como ler o gráfico:** Quanto menor a barra, melhor. Barras altas significam que o processo demorou uma eternidade no sistema (geralmente porque ficou mofando nas filas de espera).

TROCAS DE CONTEXTO

- **O que é:** É a quantidade de vezes que a CPU foi forçada a ejetar um processo e carregar outro. Isso não é de graça: o SO perde tempo salvando registradores e limpando caches. Chamamos essa lentidão de custo de overhead.
- **Como ler o gráfico:** Quanto menor a barra, melhor. Barras gigantes (como a do Round-Robin) indicam que a CPU "engasgou" muito e passou mais tempo se reorganizando do que executando tarefas úteis.

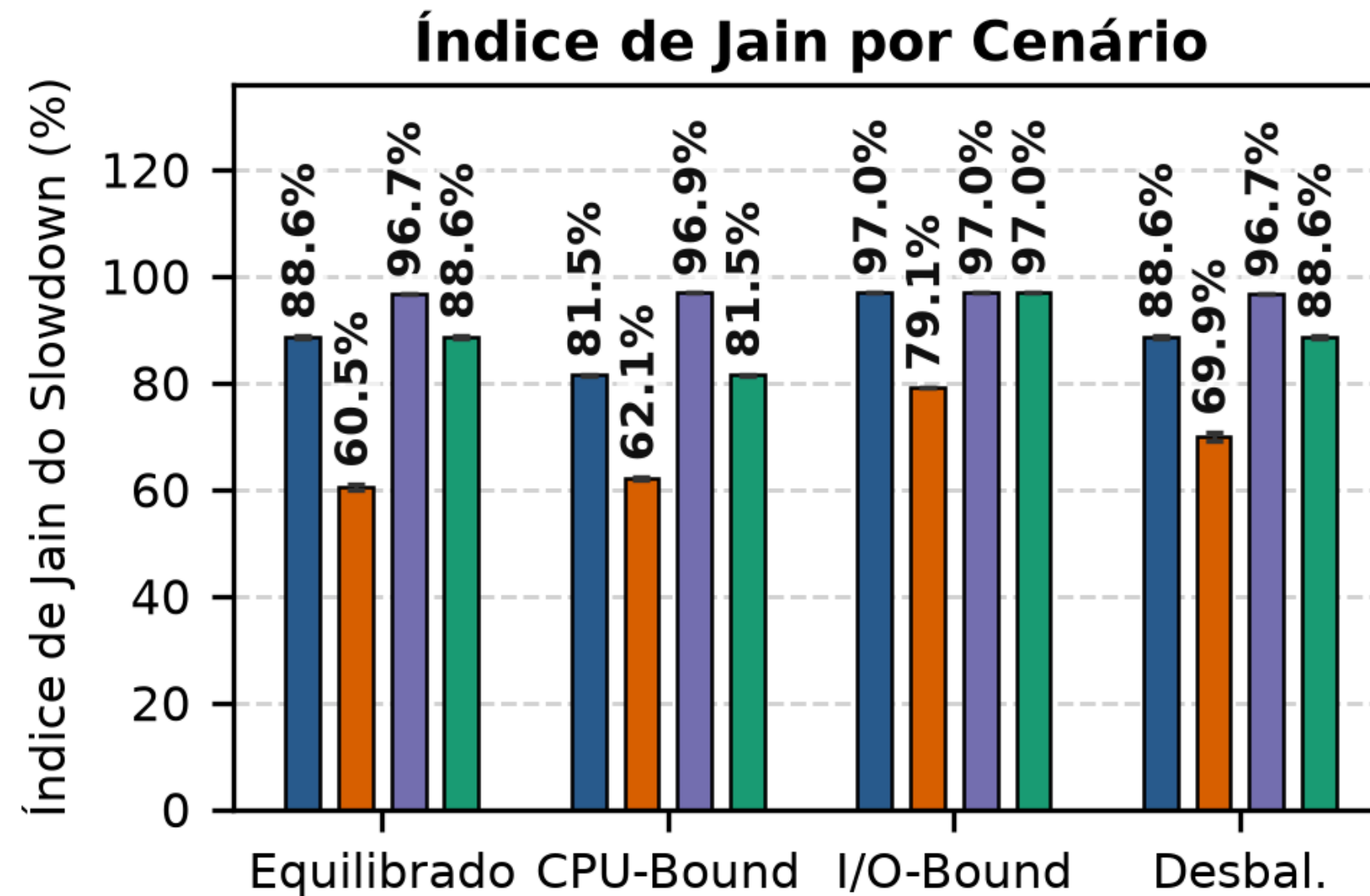
EXTRA: DICIONÁRIO DE MÉTRICAS

ÍNDICE DE JAIN

- **O que é:** É uma fórmula matemática que mede a "Igualdade/Fairness" do sistema. Ele não olha quem foi mais rápido, mas sim se os atrasos foram proporcionais para todos.
- **Como ler o gráfico:** Varia de 0 a 100%. Quanto maior, melhor. Se o valor for alto (perto de 90% ou 100%), significa que o sistema é justo. Se o gráfico despencar (como na Prioridade Pura caindo pra 69%), é a prova do crime de que ocorreu Inanição (Starvation): os figurões rodaram rápido, mas os processos fracos sofreram absurdamente, derrubando o índice de justiça.

RESULTADOS - JUSTIÇA VS. SOBRECARGA (ONDE O TEJ GANHA)

Slide: 12

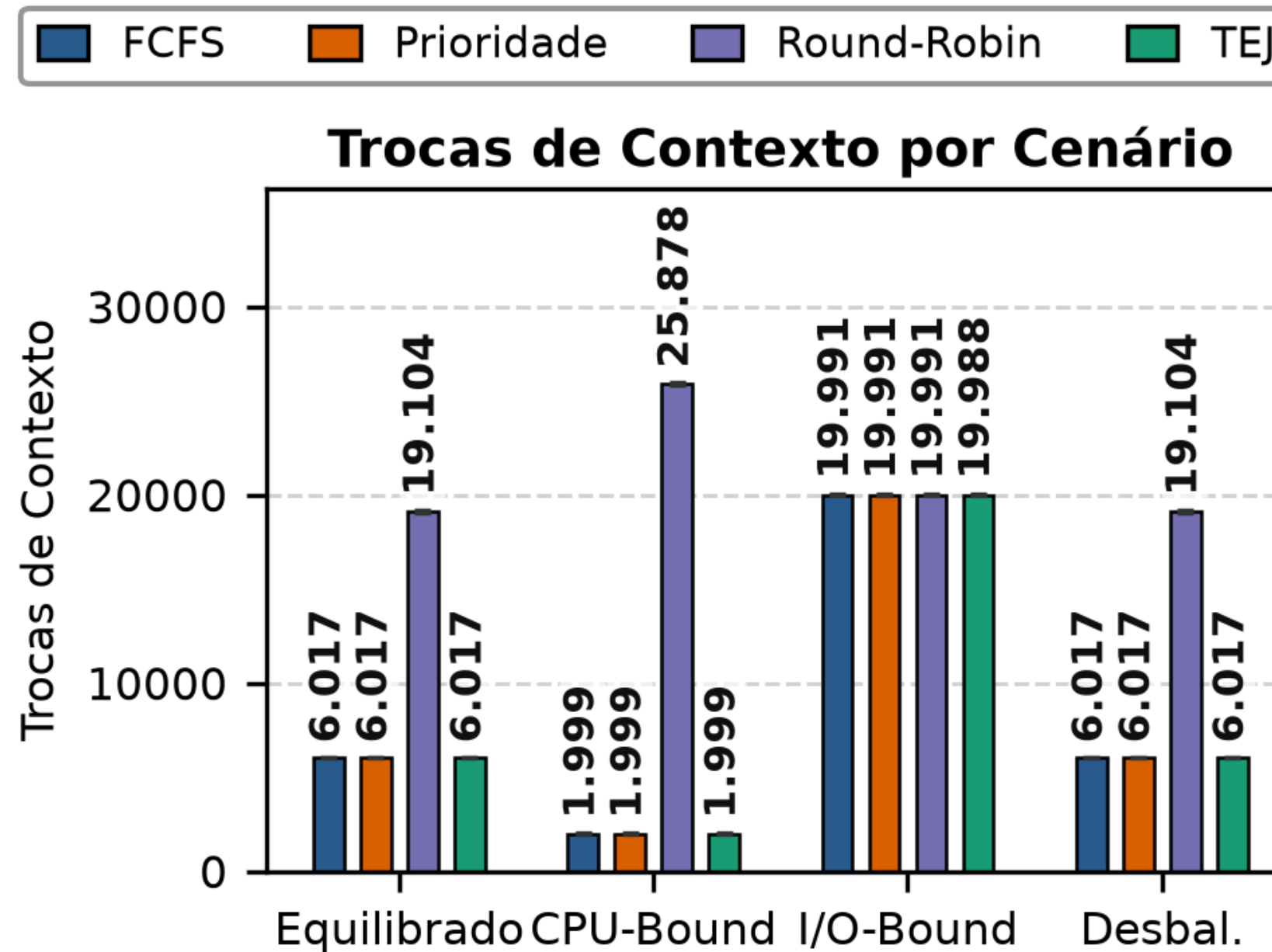


Obs: Como criamos o TEJ para resolver a inanição, estamos comparando-o com o algoritmo de Prioridade, pois ele é o principal causador desse problema.

“O algoritmo de **Prioridade Pura** tem a vantagem teórica de focar no que importa, mas a barra dele no cenário **Desbalanceado**. A justiça despencou para 69% porque os processos comuns morreram de fome (inanição). Olhem a barra do nosso TEJ ao lado: ele atuou resgatando as tarefas e salvou a justiça do sistema (88,6%).”

RESULTADOS - JUSTIÇA VS. SOBRECARGA (ONDE O TEJ GANHA)

Slide: 13



"Por outro lado, o **Round-Robin (RR)**, que desempenha bem no JAIN, tenta ser justo dando fatias de tempo iguais para todos (preempção). Mas. **os custos explodiram para mais de 25.000 trocas de contexto**. A máquina travou só tentando organizar a fila. O TEJ, sendo não-preemptivo, garantiu a justiça sem esse overhead absurdo, se mantendo lá embaixo em 1.999 trocas."

RESULTADOS - JUSTIÇA VS. SOBRECARGA (ONDE O TEJ GANHA)

Slide: 14

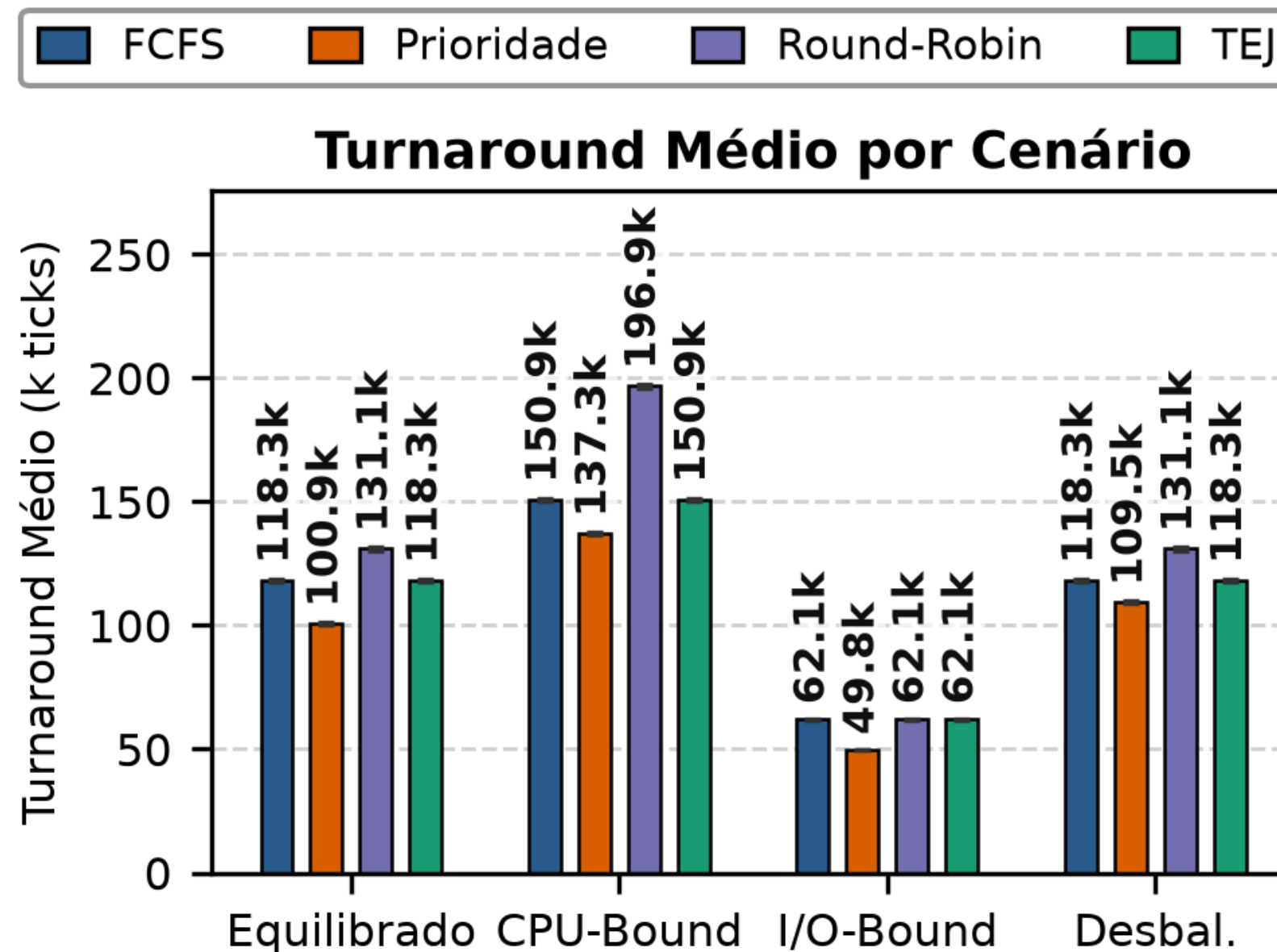
TEJ é Não Preemptivo. Ou seja, uma vez que um processo entra na sala do médico (CPU), o TEJ não chuta ele de lá de dentro até que ele termine sua rajada.

Por que escolhemos isso? Se quiséssemos interromper as tarefas, acabaríamos construindo um Round-Robin. E como provamos nos gráficos, a preempção causa uma explosão de Trocas de Contexto (pedágios para salvar registradores na memória). O nosso objetivo com o TEJ era **garantir justiça** e **evitar a inanição** mexendo apenas na fila da recepção, mas mantendo a eficiência pesada e limpa dentro do processador."

RESULTADOS: O PREÇO DA JUSTIÇA (ONDE O TEJ PERDE)

Não Existe Bala de Prata!

- O **FCFS** e a **Prioridade** têm a vantagem de serem **mais 'rápidos'** de forma global. Eles atingiram tempos de vida da tarefa (Turnaround) menores, justamente porque não pausam a fila VIP para ajudar processos atrasados."
- "O TEJ paga o seu pedágio exatamente aqui. Ao parar tudo para resgatar os processos de baixa prioridade esquecidos, nós atrasamos algumas tarefas VIP."
- "No cenário caótico, o nosso **Turnaround subiu para 118 mil ticks**, ficando atrás da Prioridade (109 mil). Mas essa é a nossa principal defesa: é um trade-off calculado. Escolhemos perder uma fração de velocidade bruta para erradicar a inanição sem estourar o limite de trocas de contexto da máquina."



MUITO OBRIGADO

Obrigado pelo seu tempo

Conselho do Processo vítima da Inanição



"Se depender do escalonador, eu vou ficar **só o osso**, por isso eu digo: **Descansa somente em Deus, ó minha alma, porque dele vem a minha esperança.**"

Salmo 62:5